

TP TACS 2026-C2 “404 Sol not found”



Introduccion

El objetivo del TP es desarrollar una aplicación que permita a los usuarios planificar actividades junto a sus amigos y conocer el estado del clima para poder concretarlas.

La aplicación funcionará de modo stand alone, y estará publicada en la nube para ser accedida.

El TP consta de diversas entregas en las cuales de forma iterativa e incremental se irán agregando funcionalidades a la aplicación.

Recomendaciones Generales

- Antes de comenzar a codificar, acordar en el equipo las tecnologías, lenguajes, frameworks, etc.
- Dividir en forma clara en el equipo las historias de cada entrega para atacarlas en paralelo donde sea posible.
- Utilizar alguna herramienta para la gestión de tareas (Scrummy, Trello, Issues de Github)
- Para bajar el riesgo de las futuras entregas aprovechar el tiempo de entregas anteriores para investigar las tecnologías.
- De ser necesario utilizar al ayudante como facilitador, en cuestiones técnicas y de organización → El rol del ayudante NO es simplemente el de corregir, sino dar soporte al equipo durante todo el proceso en cuestiones técnicas y metodológicas.

Uso de IA

El uso de herramientas de IA no solo no está prohibido, sino que recomendamos activamente su utilización durante el desarrollo del TP, tanto para explorar alternativas como para destrabar problemas, validar decisiones de diseño, acelerar la implementación o mejorar la calidad de los entregables.

Sin embargo, todo lo producido y entregado es responsabilidad exclusiva del grupo. Esto implica que cada integrante deberá comprender, validar y poder explicar cada decisión tomada, cada diseño propuesto y cada línea de código presentada, independientemente de haber utilizado asistencia de IA.

Adicionalmente, el grupo deberá documentar en el README.md/Wiki qué herramientas de IA utilizó, de qué forma las utilizó y para qué tipo de tareas fueron empleadas. No buscamos una enumeración exhaustiva de prompts, sino una descripción clara y honesta del uso realizado y del criterio con el que se integró la asistencia de IA al proceso de desarrollo.

Objetivo de la aplicación

La aplicación tiene como objetivo permitir a los usuarios organizar actividades al aire libre (un asado, un partido, una salida a la plaza, una corrida) indicando lugar, fecha y horario, e invitar a otros usuarios a sumarse a las mismas.

Cada actividad se mantiene bajo observación: la aplicación consulta periódicamente el pronóstico del lugar y horario elegidos y, si las condiciones no cumplen con los criterios definidos para esa actividad en cierto momento, notifica a todos los participantes con la anticipación configurada.

Cuando el pronóstico es desfavorable, la aplicación abre automáticamente una votación entre

los participantes proponiendo fechas y horarios alternativos, elegidos automáticamente por buen clima dentro de los días y franjas horarias permitidos por el organizador. Según el resultado de la votación, la actividad se reprograma, se mantiene o finalmente se cancela.

En etapas posteriores la aplicación incorporará funcionalidades avanzadas como reglas de clima personalizadas por tipo de actividad, sugerencias automáticas de los mejores días para una actividad, historial de reprogramaciones y alertas (notificaciones) ante cambios bruscos del pronóstico.

La aplicación debe poder escalar para manejar gran cantidad de usuarios concurrentes y actividades simultáneas bajo monitoreo. Los dueños del proyecto buscan una solución moderna, sin colas virtuales ni mecanismos manuales.

User Stories

1. Como usuario quiero crear una actividad, especificando:
 - a. Título y descripción
 - b. Tipo de actividad (aire libre, techada, mixta)
 - c. Ubicación (ciudad o coordenadas)
 - d. Fecha y horario propuestos
 - e. Cantidad mínima y máxima de participantes
2. Como organizador quiero definir las condiciones de clima aceptables para mi actividad (por ejemplo: probabilidad de lluvia máxima, temperatura mínima y máxima, viento máximo).
3. Como organizador quiero definir la ventana de anticipación con la que quiero ser avisado si el clima va a estar mal (por ejemplo: 24 horas antes).
4. Como organizador quiero definir el rango de reprogramación permitido para mi actividad (por ejemplo: hasta 3 días después, entre las 10 y las 20 hs).
5. Como usuario quiero poder buscar actividades disponibles, aplicando filtros por tipo, ubicación, fecha, etc.
6. Como usuario quiero sumarme o bajarme de una actividad mientras haya lugar.
7. Como usuario quiero ver el estado actual del clima y el pronóstico para el lugar y horario de una actividad en la que participo.
8. Como participante quiero recibir una notificación cuando el pronóstico de una actividad en la que estoy anotado no cumple con las condiciones definidas.
9. Como organizador quiero que, ante un pronóstico desfavorable, se abra una votación con alternativas de fecha y horario propuestas automáticamente por la aplicación en función del buen clima, o por mi manualmente, dentro del rango permitido.
10. Como participante quiero votar una de las alternativas propuestas y ver el resultado parcial de la votación.
11. Como organizador quiero que la actividad se re programe automáticamente a la alternativa más votada al cerrarse la votación, o que se cancele si ninguna alternativa alcanza el quórum mínimo.
12. Como usuario quiero ver mis actividades organizadas, aquellas a las que me sumé, las votaciones abiertas y el estado de cada actividad (propuesta, confirmada, reprogramada, cancelada, finalizada).
13. Como usuario quiero recibir notificaciones cuando una actividad de mi interés esté por comenzar, cuando se re programe o cuando se cancele.
14. Como administrador quiero ver estadísticas de uso y actividad de la plataforma

(actividades creadas, reprogramadas, canceladas por clima, consultas al servicio de pronóstico, etc.).

Requerimientos no funcionales

Los requerimientos no funcionales no solo son importantes para aprobar el TP sino que están directamente relacionados con la filosofía y objetivos de la materia. La calidad no se negocia.

Técnicos

- No es el objetivo del TP trabajar sobre la creación o autenticación de usuarios. Dicho esto, es importante poder diferenciar de alguna forma los mismos para poder atacar los casos de uso.
- Se debe utilizar Github/GitLab como SCM.
- Los tests son parte del código. Un caso de uso que no está debidamente testeado, tampoco está completo.
- La lógica de evaluación del clima, la generación de alternativas y la resolución de las votaciones debe poder testearse sin depender del servicio externo de pronóstico.
- Todos los métodos no triviales deben tener su correspondiente doc (ej: javadoc) explicando su función, forma de uso y cualquier otra información relevante.
- Se debe incluir en el README.md/Wiki cómo levantar la aplicación y cualquier decisión respecto del código o las soluciones utilizadas.
- La aplicación debe ser capaz de correrse desde Maven/Gradle/SBT/Node, el comando a correr debe iniciar la aplicación dentro de un Docker container.
- Se debe usar docker-compose para definir el conjunto de aplicación + db + network de modo tal que se pueda correr todo con un solo comando. Esto es obligatorio para modo local, si en la nube se va a utilizar alguna SaaS DB, entonces para el deploy solo es necesario el Docker container de la main app.
- La APP tiene que cumplir con requerimientos mínimos de seguridad (manejo de contraseñas, recursos externos, etc.)
- La aplicación consumirá un servicio externo de clima y pronóstico. Las keys nunca deben estar versionadas (commiteadas en el repo): se disponibilizarán las keys a la aplicación por configuración o variables de entorno.
- Se debe contemplar el uso responsable del servicio externo (caché, límites de requests, manejo de errores e indisponibilidad del proveedor). La aplicación no puede quedar inutilizable si el servicio de clima no responde.
- La aplicación debe soportar un load test, se utilizara alguna tool como Vegeta, Wrk, etc.
- La aplicación debe ser portable, requiriendo solamente de Gradle/Maven/SBT/Node/etc + Docker para su prueba y evaluación.
- La aplicación debe tener una interfaz de usuario fácil de utilizar, a elegir entre frontend o integración por telegram.

UI

- Si bien se espera algo sencillo, la aplicación debe tener un frontend amigable
- Utilizar algun framework CSS (shadcn, etc)

Condiciones para promoción

- Entregas en fecha
- Realizar frontend e integración con telegram

Entregas

- Las entregas deberán realizarse el día pactado antes de las 19 Hs. con un tag en el repositorio llamado Entrega_XX correspondiente al número de entrega.
- Las entregas se realizarán indicando el link al repositorio y el tag para la entrega.
- Todo retraso en una entrega que no haya sido correctamente comunicado y justificado tendrá como penalización el agregado de nuevos requisitos para la aprobación final del TP.

Entrega 1 - Esqueleto aplicación + AI setup

Esqueleto de la aplicación web, con el modelo de actividades, participantes y reglas de clima resuelto en memoria. Se deben definir las rutas REST y una forma de documentación (openapi recomendable).

Si existe, se debe definir el setup de IA a utilizar para encarar el desarrollo del TP: harness, models, CLIs, UIs, ejemplos de prompts, ADRs para decisiones de arquitectura indicando por qué fueron tomadas, docs.

Para esta entrega sí es necesario que la app corra dentro de Docker.

Entrega 2 - UI + persistencia con DB

Se debe poder interactuar con la app mediante la interfaz elegida (frontend).

Se debe desarrollar persistencia utilizando una base de datos. Se debe modificar la aplicación para que en vez de almacenar los datos en memoria, la misma lo haga utilizando alguna base a definir por el equipo.

Nota: A fines pedagógicos se solicita que la base de datos sea NoSQL.

Entrega 3 - Cloud

Para esta entrega la aplicación debe estar deployada en la nube de forma portable.